

Regularization

1. Purpose

- **Problem Addressed:** Regularization is the primary technique used to reduce **High Variance (Overfitting)**.
- **Alternative:** While getting more training data also reduces variance, it is often expensive or impossible. Regularization allows you to prevent overfitting without needing more data.

2. Regularization in Logistic Regression

To implement regularization, we modify the Cost Function J by adding a penalty term for the magnitude of the parameters.

- **Standard Cost Function:**

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

- **Regularized Cost Function (L2):**

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

- λ (**Lambda**): The **Regularization Parameter**. It is a hyperparameter tuned using the Dev Set. It controls the trade-off between fitting the training data and keeping weights small.
- $\|w\|_2^2$: The squared Euclidean norm (L2 norm) of the parameter vector w : $\sum_{j=1}^{n_x} w_j^2$.
- **Note on b :** We typically do *not* regularize the bias term b . w contains the vast majority of parameters; regularizing b (a single number) has negligible impact.

L2 vs. L1 Regularization

- **L2 Regularization (Most Common):**

- Uses the squared term $\|w\|^2$.
- Prevents overfitting by penalizing large weights.

- **L1 Regularization:**

- Adds the term $\frac{\lambda}{m} \|w\|_1$ (sum of absolute values).
- **Result:** Results in **sparse** models where many entries in w become exactly zero.
- **Use Case:** Sometimes used for model compression, but in practice, it is rarely used in deep learning compared to L2.

3. Regularization in Neural Networks

For a Neural Network, the cost function must sum the penalties for the weight matrices W across all layers L .

- **Cost Function:**

$$J(W, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L ||W^{[l]}||_F^2$$

- **Frobenius Norm ($|| \cdot ||_F$):**

- Because W is a matrix, we use the Frobenius norm instead of the standard vector L2 norm.
- It is simply the sum of the squares of all elements in the matrix:

$$||W^{[l]}||_F^2 = \sum_{i=1}^{n^{[l]}} \sum_{j=1}^{n^{[l-1]}} (w_{ij}^{[l]})^2$$

4. Gradient Descent with Regularization (Weight Decay)

Regularization changes the derivative calculation during backpropagation, leading to a concept known as "Weight Decay."

- **New Derivative:**

$$dW^{[l]} = (\text{from backprop}) + \frac{\lambda}{m} W^{[l]}$$

- **Update Rule:**

$$W^{[l]} := W^{[l]} - \alpha dW^{[l]}$$

- **Weight Decay Interpretation:**

If we substitute the new derivative into the update rule, we get:

$$W^{[l]} := W^{[l]} - \alpha [(\text{backprop}) + \frac{\lambda}{m} W^{[l]}]$$

$$W^{[l]} := \left(1 - \frac{\alpha\lambda}{m}\right) W^{[l]} - \alpha(\text{backprop})$$

- **The Term $(1 - \frac{\alpha\lambda}{m})$:** This coefficient is slightly less than 1.
 - **Mechanism:** On every step, the algorithm shrinks (decays) the weight matrix $W^{[l]}$ slightly before subtracting the gradient. This is why L2 Regularization is frequently called **Weight Decay**.
- **Python Implementation Note:** `lambda` is a reserved keyword in Python. In code, it is common to use the variable name `lamdb`.
-

Why Regularization Reduces Overfitting

1. Intuition 1: Simplifying the Network

Regularization penalizes weight matrices from being too large. Increasing the regularization parameter λ forces the weights W to decrease.

- **Mechanism:** If λ is extremely large, the optimization algorithm is incentivized to set the weight matrices W close to zero.
- **Effect:** This effectively "zeroes out" or significantly dampens the impact of many hidden units.
- **Result:** The neural network behaves like a much smaller, simpler network.
 - A complex, deep network (prone to **High Variance**) is simplified toward a linear classifier (prone to **High Bias**).
 - By tuning λ to an intermediate value, we achieve a "Just Right" balance, reducing overfitting without simplifying the network to the point of underfitting.

2. Intuition 2: Linearity of Activation Functions

This intuition focuses on how small weights force activation functions into their linear regimes, preventing the network from modeling overly complex, non-linear decision boundaries.

- **Assumption:** The network uses the **tanh** activation function, $g(z) = \tanh(z)$.
- **Logic Chain:**
 1. **High λ :** Forces parameters W to be small.
 2. **Small z :** Since $z = Wa + b$, if W is small, z will assume a small range of values (centered near zero).
 3. **Linear Regime:** The **tanh** function is roughly linear when z is close to 0.
 4. **Network Behavior:** If every layer's activation is roughly linear, the entire deep network approximates a **linear network**.
- **Conclusion:** A network operating in the linear regime cannot model the complex, high-degree non-linearities required to overfit the data. It produces a smoother, simpler decision boundary.

3. Implementation Tip: Debugging Gradient Descent

When using regularization, standard debugging practices for Gradient Descent must be adjusted.

- **Standard Check:** Plotting the Cost Function J vs. Number of Iterations to ensure monotonic decrease.
- **The Trap:** If you plot only the original loss term (sum of squared errors), it may **not** decrease monotonically.
- **The Solution:** You must plot the **full Regularized Cost Function**, which includes the penalty term:

$$J_{new} = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} ||W||_F^2$$

Dropout Regularization

1. Conceptual Overview

Dropout is a powerful regularization technique used to address **High Variance (Overfitting)**. Instead of modifying the cost function (like L2 regularization), it modifies the network architecture itself during training.

- **Mechanism:** For each training example and each iteration, the algorithm goes through every layer and randomly eliminates (sets to zero) a subset of nodes based on a specified probability.
- **Effect:** The network becomes a smaller, "diminished" version of itself for that specific step. Because the dropped nodes change with every iteration, no single neuron can rely too heavily on any specific input feature, forcing the network to learn more robust features.

2. Implementation: Inverted Dropout

Inverted Dropout is the standard modern implementation. It ensures that the scale of the activations remains consistent between training and testing, simplifying the test-time process.

Algorithm (Example for Layer $l = 3$)

1. Create a Mask Vector ($d^{[3]}$):

Generate a random matrix of the same shape as the activation matrix $a^{[3]}$.

- `d3 = np.random.rand(a3.shape[0], a3.shape[1]) < keep_prob`
- **keep_prob:** A hyperparameter (e.g., 0.8) representing the probability that a unit is **kept**.

- **Result:** A boolean mask (or 1s and 0s) where 1 means "keep" and 0 means "drop".

2. Apply the Mask:

Multiply the activations by the mask to zero out the dropped units.

- `a3 = np.multiply(a3, d3)`
- *Note:* In Python, multiplying by Booleans treats True as 1 and False as 0.

3. Scale the Activations (The "Inverted" Step):

Divide the remaining activations by `keep_prob`.

- `a3 /= keep_prob`
- **Why is this necessary?**
 - If `keep_prob = 0.8`, you are removing ~20% of the activations.
 - This reduces the expected value of the next layer's input $z^{[4]}$ by 20%.
 - Dividing by 0.8 scales the remaining values up, restoring the expected value of $a^{[3]}$ to its original magnitude. This ensures that the scale of data passing through the network doesn't shrink.

3. Dropout at Test Time

When making actual predictions after training, the procedure changes significantly:

- **No Randomness:** Do **not** apply dropout. You want the output to be deterministic, not noisy.
- **No Scaling Needed:** Because **Inverted Dropout** (scaling by `1/keep_prob`) was applied *during training*, the weights are already learned at the correct scale. You can simply perform standard forward propagation using all neurons.
 - *Historical Note:* Older versions of dropout did not scale during training and required scaling weights at test time. This is less common now.

4. Key Implementation Details

- **Per-Iteration Randomness:** A new random mask is generated for every training example (or mini-batch) and every iteration of gradient descent.
 - **Backpropagation:** The same mask d used in forward propagation must be applied during backpropagation to ensure gradients are only calculated for the active nodes.
-

Understanding Dropout

1. Intuition: Why does it work?

Beyond the idea of training a "smaller" network, there is a deeper intuition regarding feature reliance.

- **The "Unreliable Partner" Perspective:**
 - A single hidden unit cannot rely on any specific input feature because that feature could be randomly eliminated at any step.
 - **Consequence:** The unit is forced to spread its weights out across all inputs rather than putting all its "bets" on one specific input.
- **Weight Shrinkage:**
 - By spreading the weights, the squared norm of the weight vectors tends to shrink.
 - **Connection to L2:** Dropout effectively acts as an adaptive form of **L2 Regularization**. It creates a similar effect of penalizing large weights to prevent overfitting.

2. Tuning `keep_prob` by Layer

You do not need to apply the same `keep_prob` (probability of keeping a unit) to every layer. You can vary it based on the parameter density of each layer.

- **Dense Layers (High Overfitting Risk):**
 - *Example:* A layer with a 7×7 weight matrix (lots of parameters).
 - *Setting:* Set a **lower** `keep_prob` (e.g., 0.5) to apply stronger regularization.
- **Sparse/Smaller Layers (Low Overfitting Risk):**
 - *Example:* A layer with fewer parameters.
 - *Setting:* Set a **higher** `keep_prob` (e.g., 0.7 or 1.0).
- **Input Layer:**
 - *Setting:* Usually set `keep_prob` to **1.0** (no dropout).
 - *Exception:* Occasionally set to ~0.9, but eliminating input features is rare in practice.
- **Trade-off:** Varying `keep_prob` by layer allows precise control but significantly increases the number of **hyperparameters** you must tune via cross-validation.

3. Use Case: Computer Vision

- **Context:** Computer Vision (CV) models process huge input sizes (pixels) and often lack sufficient data relative to the model size.
- **Prevalence:** Because CV models are highly prone to overfitting, Dropout is almost a standard "default" technique in this field.
- **Caveat:** Dropout is fundamentally a **regularization technique**. If your network is *not* overfitting (common in other domains), using Dropout may be unnecessary or even detrimental.

4. Downside: Debugging Gradient Descent

A major disadvantage of Dropout is that the Cost Function J becomes stochastic and ill-defined.

- **The Problem:** Because nodes are randomly removed, the cost function changes on every iteration. You lose the ability to plot a clean, monotonically decreasing graph of J vs. Iterations to verify convergence.
 - **The Solution (Workaround):**
 1. **Turn Dropout Off:** Set `keep_prob = 1.0` temporarily.
 2. **Verify:** Run the code and confirm that J decreases monotonically.
 3. **Turn Dropout On:** Re-enable the dropout parameters for actual training.
-

Other Regularization Methods

1. Data Augmentation

Concept: artificially increasing the size of the training set by generating new "fake" examples from existing data. This is a cheap and effective way to reduce variance (overfitting) when collecting new labeled data is too expensive.

- **Techniques (Computer Vision):**
 - **Mirroring:** Flipping images horizontally.
 - **Random Cropping:** Taking random zooms or translations of the original image.
 - **Rotation/Distortion:** Rotating or warping images (common in OCR for digits).
- **Key Insight:** While these synthesized examples are not as informative as brand-new independent data points (since they introduce redundancy), they

tell the algorithm that specific changes (e.g., flipping a cat) do not change the class label.

2. Early Stopping

Concept: Stopping the training process *before* the training error converges, specifically when the **Dev Set error** begins to increase.

- **Mechanism:**

1. Monitor both Training Error and Dev Set Error during Gradient Descent.
2. Training Error decreases monotonically.
3. Dev Set Error usually decreases initially, then flattens, and eventually starts increasing (indicating overfitting).
4. **Action:** Stop training at the point where Dev Set Error is lowest.

- **Why it works (Connection to L2 Regularization):**

- Parameters W are initialized to small random values (near zero).
- As training progresses, W grows larger.
- Stopping early restricts W to a "mid-size" value, preventing parameters from growing large enough to overfit.

3. Early Stopping vs. L2 Regularization

- **The Downside of Early Stopping (Orthogonalization):**

- Ideally, ML involves two independent tasks:
 1. **Optimize J :** Drive the cost function as low as possible (e.g., using Gradient Descent).
 2. **Regularize:** Prevent overfitting (e.g., using L2, Dropout).
- **Coupled Tasks:** Early stopping mixes these two. By stopping early, you interfere with the optimization of J (Task 1) to solve overfitting (Task 2). You never fully optimize the cost function.

- **Comparison:**

- **L2 Regularization:** Allows you to train as long as possible (optimizing J fully) while controlling overfitting separately. *Disadvantage:* Requires training multiple models to search for the optimal hyperparameter λ (Computationally expensive).
 - **Early Stopping:** Efficient. You find a "mid-size" W in a single training run without needing to search for λ . *Disadvantage:* Violates orthogonalization; does not fully optimize the cost function.
-